

未命名文档

结合 `PR_Gate.yml`（任务编排）和 `Variables_PR.yml`（全局配置），可推导出这是一套 **Azure DevOps PR构建门控流程**，核心围绕「PR提交后的自动化构建、测试、质量检查」展开，流程高度可配置（通过开关变量控制环节启停），以下是分阶段的完整推导：

一、核心前置：全局变量配置（Variables_PR.yml）

所有流程的基础配置由该文件定义，核心维度如下：

配置类别	关键内容
代理/环境	指定构建代理池（ <code>CT_CN_WorkSpace_PR</code> ）、代理需求（ <code>CTWS_PR_Build</code> ）、TFS/Artifactory地址
任务开关（核心）	全局控制各环节是否执行（如 <code>Build: true</code> 启用构建， <code>TICS/Resharper: false</code> 关闭质量检查）
路径/组件	组件名（ <code>CT_Serviceability</code> ）、解决方案路径、测试DLL路径、NuGet包路径、脚本路径
工具配置	MSBuild（VS2022）、NUnit测试适配器、TICS/Resharper/CodeScene等工具的URL/脚本路径
门控豁免	<code>ccpassgates/ticspassgates</code> 等变量控制是否跳过代码合规/质量门控检查

二、PR构建门控主流程（PR_Gate.yml）

该文件是Azure DevOps Pipeline的任务编排层，通过「模板调用+条件分支」实现流程编排，整体执行逻辑如下：

阶段1：参数初始化 & 环境准备

- 接收自定义入参：代理能力（agentcapability）、组件名（ComponentName）、版本（Version）、测试开关（RunModuleTest，默认false）、测试目标文件夹等；
- 加载 Variables_PR.yml 全局变量：确定代理池、构建开关、工具路径、组件路径等基础配置。

阶段2：门控异常预处理（必执行）

调用 GatedException.yml 模板，核心逻辑：

- 校验PR构建的「门控豁免规则」（结合 ccpassgates/ticspassgates 等变量）；
- 处理构建前的异常场景（如跳过特定检查、豁免临时合规问题）；
- 传递核心参数：代理配置、组件名、解决方案路径（ExtInf/\${ComponentName}Inf.sln + Src/\${ComponentName}Impl.sln）。

阶段3：核心构建（受 Build: true 控制）

虽未在 PR_Gate.yml 直接体现，但 Variables_PR.yml 中 Build: true 且所有后置任务依赖 Build，因此核心构建逻辑为：

- 用VS2022的MSBuild构建指定解决方案（Inf.sln / Impl.sln）；
- 构建配置：Any CPU 平台 + Release 编译模式；
- 依赖拉取：从 3rdPartyPackagesPath 加载第三方包，可选执行「第三方依赖检查」（3rdPartyRefCheck: false 暂关闭）。

阶段4：单元测试（条件分支，受开关控制）

根据 RunModuleTest 参数选择测试模板，核心逻辑：

条件	执行模板	测试类型	补充配置（来自 Variables）
RunModuleTest=true	Module_Nunit_PR.yml	模块级NUnit测试	测试适配器路径、测试DLL路径、最大失败重试3次
RunModuleTest=false（默认）	Nunit_PR1.yml	常规NUnit单元测试	最大失败重跑阈值5个用例、Unit_Testing: false 暂关闭

阶段5：后置质量检查（依赖Build完成，均暂关闭）

依次调用3个质量检查模板，均依赖 Build 任务完成，且受全局开关控制：

模板名	检查类型	开关状态 (Variables)	核心逻辑
TICS1.yml	TICS代码质量分析	TICS: false	调用 TICS_ABS_Count_Gates.ps1 检查代码复杂度、重复率、合规性
ResharperPR1.yml	Resharper静态代码分析	Resharper: false	检查代码风格、潜在 Bug、性能问题，支持门控豁免 (resharpassgates)
ProjectSettings_PR1.yml	项目配置合规检查	Project_Settings: false	校验编译选项、编译器警告、项目文件格式等 (Compiler_Warnings: false)

阶段6：可选扩展流程（均暂关闭）

虽未在 PR_Gate.yml 直接调用，但 Variables_PR.yml 暴露了更多可扩展环节：

- 1. **NuGet包管理**：校验 nuspec 文件合法性 (nuspec_checker.ps1) → 上传nupkg到 Artifactory (nuget_uploader.ps1) ；
- 2. **安全/合规检查**：Coverity（代码漏洞）、Fortify（安全扫描）、SonarQube（代码质量）、BlackDuck（开源合规） ；
- 3. **代码分析**：CodeScene（代码增量分析）、PR大小检查、工作项状态校验；
- 4. **打包/部署**： Packaging/Deployment: false 暂关闭，未执行构建产物打包、部署；
- 5. **组件测试**： ComponentTests_Run: false 暂关闭模块级集成测试。

三、流程核心特点

- 1. **高度可配置**：通过布尔变量（如 TICS/Resharper ）一键启停各环节，适配不同PR的检查需求；
- 2. **依赖关系清晰**：所有质量检查均依赖 Build 任务，保证「先构建成功，再做检查」；
- 3. **门控灵活**：支持异常豁免 (passgates 变量)、PR大小旁路，适配紧急PR的快速合并；
- 4. **环境标准化**：代理池、工具路径、组件路径统一配置，降低环境不一致风险；
- 5. **可扩展性强**：预留了NuGet、安全扫描、部署等扩展点，可按需开启。

四、简化版执行链路（当前开关状态）

Code block

- 1 参数初始化 → 门控异常预处理 → 执行Build构建（VS2022 MSBuild编译）→ 执行常规NUnit测试（开关关闭，无实际测试）→ 执行TICS/Resharper/项目配置检查（均关闭）→ 流程结束

（注：当前 `Variables_PR.yml` 中大部分质量检查/测试开关为 `false`，因此实际执行的核心仅为「构建+门控异常检查」，测试和质量检查暂未生效。）

（注：文档部分内容可能由 AI 生成）